

PE-EC702A — Adaptive Signal Processing

Semester VII • 3L:0T:0P • 3 Credits

Module 5 Exam Notes: Recursive Least Squares (RLS) & Advanced Topics

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Introduction to Recursive Least Squares (RLS)	2
1.1	RLS Cost Function	2
1.2	Least-Squares Normal Equations	2
2	Vector Space Formulation of RLS and Matrix Inversion Lemma	2
2.1	Matrix Inversion Lemma (Woodbury Identity)	2
2.2	The Gain Vector	2
3	RLS Transversal Adaptive Filter Algorithm	3
3.1	RLS Lattice Filters	3
4	Advanced Topics	3
4.1	Affine Projection Algorithm (APA)	3
4.2	Subspace-Based Adaptive Filters	4
4.3	Partial Update Algorithms	4
4.4	QR Decomposition (QRD-RLS) and Systolic Arrays	4

1. Introduction to Recursive Least Squares (RLS)

The Recursive Least Squares (RLS) algorithm minimizes a deterministic, exponentially weighted sum of squared errors, rather than statistical ensemble averages.

1.1 RLS Cost Function

The cost function to minimize at time n is:

$$\mathcal{E}(n) = \sum_{i=1}^n \lambda^{n-i} |e(i, n)|^2 \quad (1)$$

where $\lambda \in (0, 1]$ is the **exponential forgetting factor** (typically $0.95 \leq \lambda \leq 1.0$), and the error signal is $e(i, n) = d(i) - \mathbf{w}^H(n)\mathbf{x}(i)$ for $i = 1, 2, \dots, n$, using the weight vector at current time n .

1.2 Least-Squares Normal Equations

Setting the gradient of $\mathcal{E}(n)$ with respect to $\mathbf{w}^*$ to zero yields the deterministic normal equations:

$$\square(n)\mathbf{w}(n) = \mathbf{z}(n) \implies \mathbf{w}(n) = \square^{-1}(n)\mathbf{z}(n) \quad (2)$$

where the deterministic correlation matrix $\square(n)$ and cross-correlation vector $\mathbf{z}(n)$ are:

$$\square(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{x}(i)\mathbf{x}^H(i) = \lambda \square(n-1) + \mathbf{x}(n)\mathbf{x}^H(n) \quad (3)$$

$$\mathbf{z}(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{x}(i)d^*(i) = \lambda \mathbf{z}(n-1) + \mathbf{x}(n)d^*(n) \quad (4)$$

2. Vector Space Formulation of RLS and Matrix Inversion Lemma

2.1 Matrix Inversion Lemma (Woodbury Identity)

To update $\square^{-1}(n)$ without direct inversion, we use the Matrix Inversion Lemma:

$$(\mathbf{A} + \mathbf{BCD})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{B}(\mathbf{C}^{-1} + \mathbf{DA}^{-1}\mathbf{B})^{-1}\mathbf{DA}^{-1} \quad (5)$$

By defining $\mathbf{P}(n) = \square^{-1}(n)$ and substituting $\mathbf{A} = \lambda \square(n-1)$, $\mathbf{B} = \mathbf{x}(n)$, $\mathbf{C} = 1$, and $\mathbf{D} = \mathbf{x}^H(n)$, we obtain:

$$\mathbf{P}(n) = \lambda^{-1}\mathbf{P}(n-1) - \frac{\lambda^{-2}\mathbf{P}(n-1)\mathbf{x}(n)\mathbf{x}^H(n)\mathbf{P}(n-1)}{1 + \lambda^{-1}\mathbf{x}^H(n)\mathbf{P}(n-1)\mathbf{x}(n)} \quad (6)$$

2.2 The Gain Vector

Defining the gain vector $\mathbf{k}(n)$ as:

$$\mathbf{k}(n) = \frac{\mathbf{P}(n-1)\mathbf{x}(n)}{\lambda + \mathbf{x}^H(n)\mathbf{P}(n-1)\mathbf{x}(n)} = \mathbf{P}(n)\mathbf{x}(n) \quad (7)$$

Allows writing the inverse correlation update compactly as:

$$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n)\mathbf{x}^H(n)\mathbf{P}(n-1)] \quad (8)$$

3. RLS Transversal Adaptive Filter Algorithm

The complete step-by-step transversal RLS algorithm is structured as follows:

RLS Algorithm Steps

Initialization (at $n = 0$):

$$\mathbf{w}(0) = \mathbf{0} \quad (9)$$

$$\mathbf{P}(0) = \delta^{-1} \mathbf{I} \quad (\delta \text{ is a small positive constant}) \quad (10)$$

Recursion (for each time index $n = 1, 2, \dots$):

1. Compute the gain vector:

$$\mathbf{k}(n) = \frac{\mathbf{P}(n-1)\mathbf{x}(n)}{\lambda + \mathbf{x}^H(n)\mathbf{P}(n-1)\mathbf{x}(n)}$$

2. Calculate the a priori estimation error:

$$\xi(n) = d(n) - \mathbf{w}^H(n-1)\mathbf{x}(n)$$

3. Update the filter weights:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi^*(n)$$

4. Update the inverse correlation matrix:

$$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n)\mathbf{x}^H(n)\mathbf{P}(n-1)]$$

3.1 RLS Lattice Filters

RLS lattice filters (Least-Squares Lattice, LSL) combine the fast convergence of least-squares algorithms with the modular, low-computational complexity $O(M)$ of lattice structures, solving numerical sensitivity issues of standard RLS.

4. Advanced Topics

4.1 Affine Projection Algorithm (APA)

APA generalizes NLMS. Instead of adjusting the weight vector based on the current input vector $\mathbf{x}(n)$, APA projects the weights onto a subspace spanned by P past input vectors (where P is the projection order).

- **Significance:** For $P = 1$, APA reduces to NLMS. For $P = M$ (filter length), it approaches RLS performance. It offers a flexible trade-off between computational complexity and convergence speed for highly correlated inputs.

4.2 Subspace-Based Adaptive Filters

These algorithms decompose the signal space into orthogonal signal and noise subspaces using eigendecomposition or singular value decomposition (SVD). By projecting updates exclusively onto the signal subspace, it filters out noise components, enhancing performance in high-resolution estimation tasks.

4.3 Partial Update Algorithms

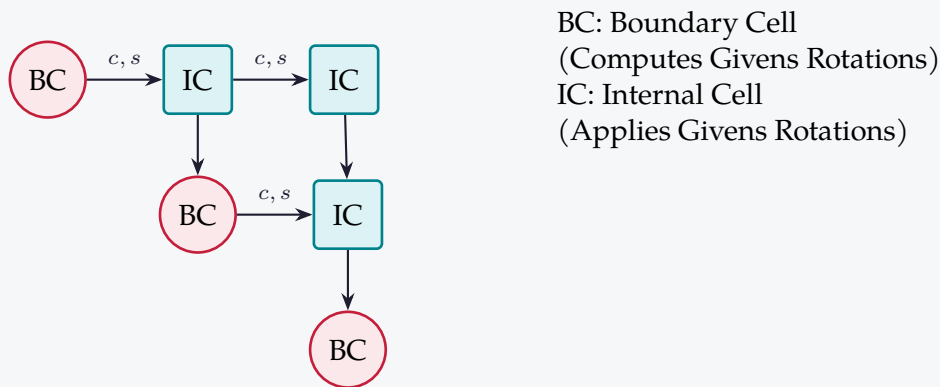
To save power and processor cycles in resource-constrained environments (e.g., mobile devices), these algorithms update only a subset of the M filter coefficients at each iteration, selecting either a fixed subset (Periodic LMS) or a dynamic subset based on magnitude (Max-NLMS).

4.4 QR Decomposition (QRD-RLS) and Systolic Arrays

Standard RLS is numerically unstable under finite word-length effects because the matrix $\mathbf{P}(n)$ can lose its positive-definiteness due to round-off errors.

- **QRD-RLS:** Solves the least-squares problem by applying unitary transformations (Givens rotations) directly to the data matrix $\mathbf{X}(n)$, decomposing it into an orthogonal matrix $\mathbf{Q}(n)$ and an upper triangular matrix $\mathbf{R}(n)$. Since unitary transformations preserve norms, QRD-RLS is numerically stable.
- **Systolic Arrays:** A systolic array is a parallel network of processors (Boundary Cells and Internal Cells) that pipelined Givens rotations for real-time QRD-RLS VLSI hardware execution.

Systolic Array Structure for QRD-RLS



Example: RLS Iteration Computation

Given $\lambda = 1.0$, the current estimate $\mathbf{w}(n-1) = [0.5, -0.2]^T$, and the inverse correlation matrix:

$$\mathbf{P}(n-1) = \begin{bmatrix} 2.0 & 0.5 \\ 0.5 & 1.0 \end{bmatrix}$$

The new input sample vector is $\mathbf{x}(n) = [1.0, 2.0]^T$, and the desired signal is $d(n) = 0.8$. Execute one step of the RLS recursion to find the updated weight vector $\mathbf{w}(n)$.

Solution:

1. Compute the numerator of the gain vector:

$$\mathbf{P}(n-1)\mathbf{x}(n) = \begin{bmatrix} 2.0 & 0.5 \\ 0.5 & 1.0 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} (2 \times 1) + (0.5 \times 2) \\ (0.5 \times 1) + (1 \times 2) \end{bmatrix} = \begin{bmatrix} 3.0 \\ 2.5 \end{bmatrix}$$

2. Compute the denominator of the gain vector:

$$\lambda + \mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{x}(n) = 1.0 + \begin{bmatrix} 1.0 & 2.0 \end{bmatrix} \begin{bmatrix} 3.0 \\ 2.5 \end{bmatrix} = 1.0 + (3.0 + 5.0) = 9.0$$

3. Compute the gain vector $\mathbf{k}(n)$:

$$\mathbf{k}(n) = \frac{1}{9.0} \begin{bmatrix} 3.0 \\ 2.5 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 5/18 \end{bmatrix} \approx \begin{bmatrix} 0.3333 \\ 0.2778 \end{bmatrix}$$

4. Compute the a priori estimation error $\xi(n)$:

$$\xi(n) = d(n) - \mathbf{w}^T(n-1)\mathbf{x}(n) = 0.8 - \begin{bmatrix} 0.5 & -0.2 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} = 0.8 - (0.5 - 0.4) = 0.7$$

5. Update the weight vector $\mathbf{w}(n)$:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi(n) = \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix} + 0.7 \begin{bmatrix} 1/3 \\ 5/18 \end{bmatrix} = \begin{bmatrix} 0.5 + 0.2333 \\ -0.2 + 0.1944 \end{bmatrix} = \begin{bmatrix} 0.7333 \\ -0.0056 \end{bmatrix}$$

Quick Revision — Module V

Recursive Least Squares (RLS) & Advanced Topics | PE-EC702A

Key Formulas at a Glance

RLS Cost Function	$\mathcal{E}(n) = \sum_{i=1}^n \lambda^{n-i} e(i, n) ^2$
Correlation Matrix Update	$\mathbf{P}(n) = \lambda \mathbf{P}(n-1) + \mathbf{x}(n)\mathbf{x}^H(n)$
Matrix Inversion Lemma	$(\mathbf{A} + \mathbf{BCD})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{B}(\mathbf{C}^{-1} + \mathbf{DA}^{-1}\mathbf{B})^{-1}\mathbf{DA}^{-1}$
Gain Vector $\mathbf{k}(n)$	$\mathbf{k}(n) = \frac{\mathbf{P}(n-1)\mathbf{x}(n)}{\lambda + \mathbf{x}^H(n)\mathbf{P}(n-1)\mathbf{x}(n)}$
Weight Update	$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\xi^*(n) \quad (\xi(n) = d(n) - \mathbf{w}^H(n-1)\mathbf{x}(n))$
Inverse Matrix Update	$\mathbf{P}(n) = \lambda^{-1} [\mathbf{P}(n-1) - \mathbf{k}(n)\mathbf{x}^H(n)\mathbf{P}(n-1)]$

One-Line Definitions

- **RLS Algorithm:** An adaptive filter algorithm that recursively finds coefficients to minimize deterministic sum of squared errors.
- **Forgetting Factor:** A scalar parameter $\lambda \leq 1.0$ weighting recent data samples more heavily than older ones.
- **Gain Vector:** The vector $\mathbf{k}(n)$ that scales the error term in the RLS weight update equation.
- **Matrix Inversion Lemma:** A mathematical formula allowing recursive computation of matrix inverses without direct inversion.
- **Systolic Array:** A network of processing elements configured to execute parallel QRD matrix operations in hardware.
- **Boundary Cell:** A processor in a QRD systolic array calculating Givens rotation parameters (cosines and sines).

Frequently Asked / Viva Questions

Q1. Compare RLS and LMS in terms of convergence speed and computational complexity.

RLS converges much faster than LMS (typically within $2M$ iterations, independent of eigenvalue spread) but requires higher computational complexity ($O(M^2)$ multiplications per sample compared to $O(M)$ for LMS).

Q2. Explain the role of the forgetting factor λ in the RLS cost function.

λ determines the memory of the filter. If $\lambda = 1.0$, the filter has infinite memory (all data weighted equally). If $\lambda < 1.0$ (typically 0.98), older data is exponentially attenuated, allowing the filter to track non-stationary changes.

Frequently Asked / Viva Questions (contd.)

Q3. Write down the Woodbury identity (Matrix Inversion Lemma) and explain its use in RLS.

$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}$. It is used to recursively update the inverse correlation matrix $\mathbf{P}(n) = \mathbf{\Phi}^{-1}(n)$ without performing an $O(M^3)$ matrix inversion step.

Q4. What are the typical initialization values for the RLS algorithm?

$\mathbf{w}(0) = \mathbf{0}$ and $\mathbf{P}(0) = \delta^{-1}\mathbf{I}$, where δ is a small positive regularization parameter (e.g., 0.01).

Q5. What is the difference between a priori error $\xi(n)$ and a posteriori error $e(n)$ in RLS?

The a priori error $\xi(n) = d(n) - \mathbf{w}^H(n-1)\mathbf{x}(n)$ is calculated using the old weight vector, while the a posteriori error $e(n) = d(n) - \mathbf{w}^H(n)\mathbf{x}(n)$ is calculated using the updated weight vector.

Q6. What is the purpose of using QR Decomposition (QRD) in RLS filters?

Standard RLS suffers from numerical instability under finite precision arithmetic, causing $\mathbf{P}(n)$ to lose positive-definiteness. QRD solves this by applying unitary rotations to the data matrix, ensuring numerical stability.

Q7. What is the Affine Projection Algorithm (APA)?

APA is an intermediate algorithm between NLMS and RLS that updates weights based on the current and $P - 1$ past input vectors, speeding up convergence for correlated signals with moderate complexity.

Q8. What are partial update algorithms and where are they useful?

They are algorithms that update only a fraction of the filter weights at each time index to reduce power consumption and processing requirements, useful in acoustic echo cancellation in mobile phones.

Q9. What are the functions of boundary cells and internal cells in a systolic array?

Boundary cells compute the Givens rotation parameters (cosine c and sine s) to zero out elements of the incoming data matrix, while internal cells apply these rotations to the rest of the matrix rows.

Q10. Why does RLS converge independently of the eigenvalue spread of the input signal?

Because RLS pre-whitens the input signal by multiplying the error vector with the inverse correlation matrix $\mathbf{P}(n)$, which de-correlates the input modes.

Comparison of LMS vs. RLS Algorithms

Parameter	LMS Algorithm	RLS Algorithm
Convergence Rate	Slow (dependent on input eigenvalue spread)	Fast (typically within $2M$ iterations, independent of spread)
Complexity	$O(M)$ multiplications per sample	$O(M^2)$ multiplications per sample
Stability	Stable if $0 < \mu < 2/\text{Tr}(\mathbf{R})$	Can suffer from numerical instability in finite precision
Parameters	Step-size μ	Forgetting factor λ , regularization δ